# From English to Regular Expressions

## An encoder-decoder with attention, trained on a verified dataset

NLP course project (text generation with encoder-decoder architectures), Université Paris Dauphine Tunis. Mohamed Ali Aoun.

> A small LSTM encoder-decoder with attention, built from scratch, turns English descriptions into correct regular expressions: **99.4%** exact match on unseen regexes, **82.6%** on unseen sentence patterns and **48 of 50** handwritten descriptions.

## 1 Task

Given a short English description of a set of text lines, generate a regex that matches exactly those lines (the whole line). For example, "lines containing 'cat' at least 2 times" gives `(.*cat){2,}.*`, and "lines that start with a capital letter and end with a number" gives `([A-Z].*)&(.*[0-9])`, where `&` means *and* and `~` means *not*. Regexes are formal, so every output can be run and checked.

## 2 Dataset: VeriRegex

A first version on the NL-RX benchmark reached 45.3% exact match, but its labels sometimes contradict their English: "'dog' at least 2 times" is labelled `.*(dog){2,}.*`, which only accepts adjacent copies and rejects "dog and dog". We therefore built **VeriRegex**. Each example comes from a **meaning tree** (13 condition types, such as contains, starts with, at least N times, before, length, combined with not, and, or). The tree yields an English description, chosen among 216 phrasing templates, and one canonical regex. An independent Python predicate evaluates the same tree, and on about 300 probe strings per example the regex and the predicate must agree: **no disagreement ever occurred**. 5,251 trees that matched every line or none were discarded.

| Split (by regex structure) | Pairs | What it tests |
|---|---|---|
| Train | 37,586 | 14,345 distinct regex structures |
| Test: new regexes | 1,884 | combinations never seen in training |
| Test: new phrasings | 1,500 | 11 sentence patterns held out of training |
| Test: handwritten | 50 | free wording, written by hand |

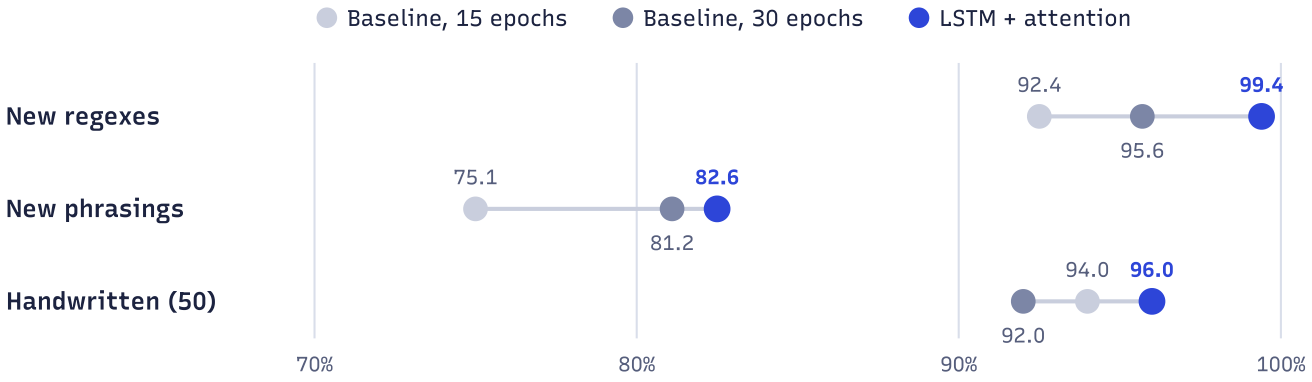
## 3 Exploratory analysis and preprocessing

Descriptions average 14 words (max 37) and regexes 17 tokens (max 67); the two lengths correlate (r = 0.78). The training vocabulary has 194 words, of which the 100 most frequent cover 91% of tokens. No test regex or description appears in training; unknown words are 0% in the new-regex test, 0.7% in the new-phrasing test and 1.2% in the handwritten set. Preprocessing: quoted literals become slots ( `__w0__` , `__w1__` , in order of appearance) in the description and the regex, and are restored after decoding, so the model handles any word; then lowercase, word tokens on the source side and character tokens on the regex side, vocabularies from training only with `<PAD>` , `<UNK>` , `<SOS>` , `<EOS>` , length-bucketed padded batches (6% padding instead of 58%), and 5% word dropout.

## 4 Model and training

Embedding (128) and a 1-layer LSTM encoder (hidden 256); its final state, through a linear and tanh bridge, initialises an LSTM decoder with hand-written additive (Bahdanau) attention over the encoder states. The baseline is the same network without attention. 1.38M parameters (0.96M without attention), cross-entropy with teacher forcing, Adam, early stopping on validation exact match; 25 minutes on a CPU for the attention model. Evaluation: exact match, behavioural equivalence on probe strings, BLEU-4, valid-regex rate.

## 5 Results




Exact match (%), greedy decoding, first attention seed; the axis starts at 70%. A second seed gives 99.7 / 83.3 / 96.0. Beam search (width 5) changes the attention model's scores by at most 0.1 point.

- **Attention is decisive for new structures.** At equal training (15 epochs) it removes nine tenths of the baseline's errors on new regexes, and six in seven against a baseline trained twice as long. The largest gain is on AND combinations: 99.4% against 92.5% for the 30-epoch baseline. It also learns far faster: 88.7% validation exact match after one epoch, against 7%.
- **Clean, interpretable outputs.** 99.6 to 100% of outputs are valid regexes, and attention maps align each slot with its quoted literal, `~` with “not” and `&` with “and”. Long descriptions such as “a regex for lines in which the text doesn’t begin with a digit and the text finishes with the substring ‘report’” give `(~([0-9].*))&(.*report)` exactly.
- **New wording is the frontier.** On held-out sentence patterns the long baseline is only 1.4 points behind: the limit there is the training language, not the architecture. Typical errors map an unseen construction onto a known one (“have X at the end” read as “starts with”, “no fewer than” read as “fewer than”).

## 6 Discussion and conclusion

The project stands out for its verified labels, rigorous out-of-distribution testing, a light from-scratch model with interpretable attention, and full reproducibility: the executed notebook regenerates the identical dataset and retrains both models to the same numbers. Its main limits are that template sentences are more regular than human writing (99.4% is an in-distribution score; the other two test sets are the honest estimates, and the handwritten set is small), that behavioural equivalence is a sampling test, and the 60-token output cap. Paraphrased training data is the natural next step. Lessons learned: check the labels before trusting a metric, and always test outside the generator’s distribution.

The notebook, the full 10-page report, the slides and a live in-browser demo accompany this summary. References: Locascio et al., EMNLP 2016; Sutskever et al., NeurIPS 2014; Bahdanau et al., ICLR 2015; Ye et al., ACL 2020.